Adaptive Traffic Signal Control using Big Data (HDFS, Hadoop, Spark) and a Cloudflare Decision Layer

Big Data Project Document: idea, architecture, decision rules and implementation plan

Prepared by: _____ **Roll no.:** _____ **Course / Guide:** _____ **Date:** October 2026

1. Project at a glance

Item	Summary
Title	Adaptive Traffic Signal Control using Big Data and an Edge Decision Model
Problem	Fixed-time traffic lights waste green time on empty roads and make long queues wait. Ambulances get stuck.
Idea	Count vehicles on every approach lane continuously, score each lane, and recommend which lanes should get green next, with a fair and safe set of rules.
Phase 1	Analyse one highway in **both directions**: flow, count, speed, peak hours. These results set the thresholds.
Phase 2	A **4-road junction with 8 approach lanes**. Every 10 seconds the system recommends red or green for each lane.
Big data stack	Kafka, **HDFS**, **Hadoop (MapReduce, YARN)**, Spark Structured Streaming, Hive
Decision layer	A **Cloudflare Worker** (edge API) that runs the decision rules and stores junction state
Special rules	Heavy-lane threshold (50 or 100 vehicles), no direct switching, skip empty lanes, never starve a lane, **ambulance gets instant priority**
Output	A recommendation `{green lanes, hold time, rule used, reason}` every 10 s, plus a dashboard and a report comparing it with a fixed-time plan

2. Problem statement and objectives

2.1 Problem statement

At most Indian junctions, signals run on a fixed timer. When one road has a very long queue and another is empty, the timer still gives both the same green time. This increases waiting time, fuel use and emissions, and it blocks emergency vehicles. We need a system that looks at the real traffic, decides which side should go next, and does so fairly and safely.

2.2 Objectives

1. Analyse a highway in both directions to understand flow, vehicle count, speed and peak periods (Phase 1).
2. Continuously track vehicle counts on all **8 approach lanes** of a 4-road junction (Phase 2).
3. Use a **fixed count threshold** (for example 50 or 100 vehicles) to detect a heavy lane. A heavy lane gets a higher priority, but the signal is **not switched directly**; signals change only on the 10-second decision tick.
4. Give green first to the lane with the **highest priority score**, and give green to its compatible opposite lane at the same time.
5. **Do not give green** to a lane with very little traffic.
6. **Never leave a lane closed for too long.** If a lane has waited a long time, open it and clear it fully, even if it has few vehicles.
7. **Ambulance priority:** an ambulance on any lane gets green immediately (at the next safe moment).
8. Process all lanes **in parallel** using HDFS, Hadoop and Spark, and make the decision with a Cloudflare edge decision model.
9. Prove the benefit by comparing against a normal fixed-time signal in simulation.

2.3 Assumptions and interpretation notes

These are the choices I made while turning the idea into a design. Change them if your guide wants something different.

- **“8 sides” = 8 approach lanes.** The junction has 4 roads (North, East, South, West). Each road has 2 approach lanes: lane 1 = through lane (also takes the left turn), lane 2 = right-turn lane (India drives on the left, so the right turn crosses opposing traffic). This gives N1, N2, E1, E2, S1, S2, W1, W2. If you meant 8 different movements, the

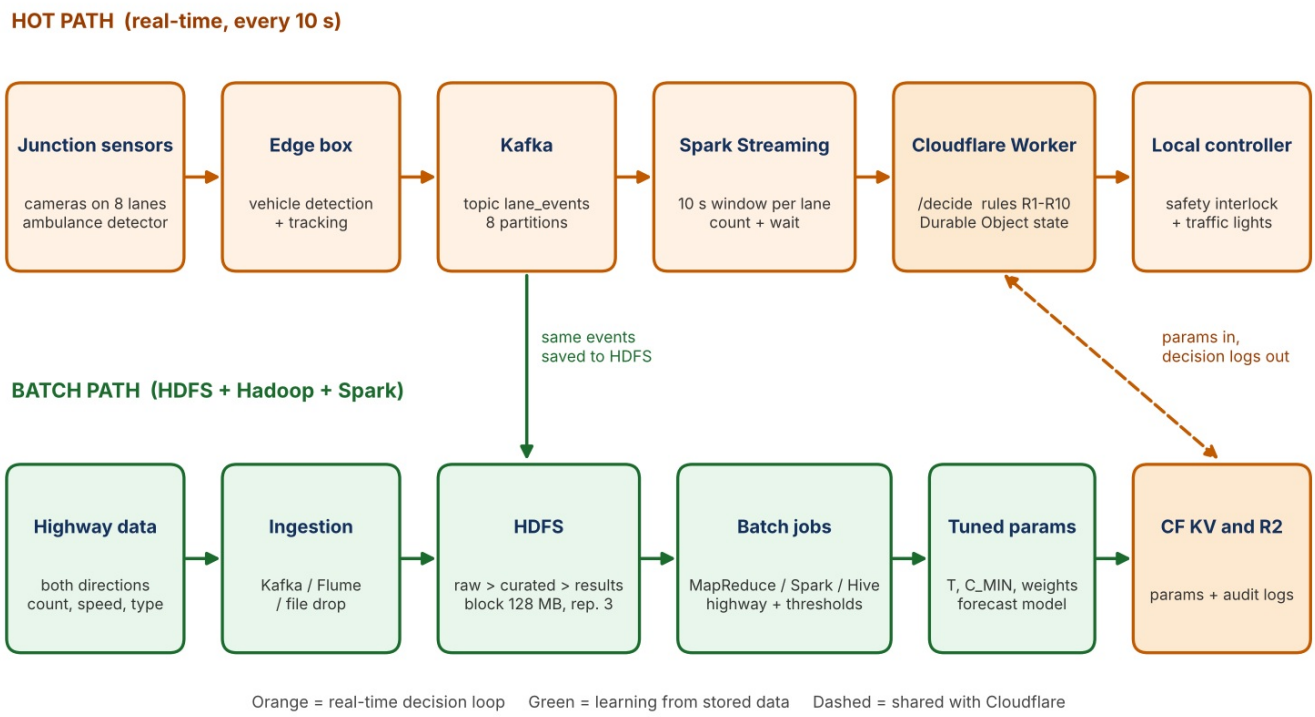
algorithm is the same; only the lane list and the compatibility table change.

- **“Cloudflare decision model” = a Cloudflare Worker** that hosts the decision logic at the edge. If you meant Claude (the AI assistant) as the decision model, the rule engine in Section 11 stays as the safety guard and the LLM can be added only as an advisor.
- **“Count” = number of vehicles currently in a lane’s monitoring zone** (for example the last 100 to 150 m before the stop line), computed from tracked vehicles in a 10-second window.
- The 50 or 100 limit is a **configurable constant** called `T`. Default is 50; use 100 for long or wide approaches. Phase 1 data is used to choose it.
- The project is a **prototype/simulation**. A real deployment needs approval and certified signal-controller hardware.

3. Scope

In scope	Out of scope
Highway flow analysis in both directions (offline, big data)	Controlling real traffic lights on a public road
One 4-road junction, 8 lanes, real-time recommendation	Network-wide coordination of many junctions (listed as future work)
Rule-based decision engine with ambulance, fairness and skip logic	Pedestrian signals and public transport priority
Simulation with SUMO or a synthetic generator, dashboard	Legal and certification work

4. System architecture



Overall architecture: the orange hot path makes the decision every 10 seconds; the green batch path stores everything on HDFS and learns better parameters.

4.1 Layers

Layer	Components	Job
Sensing	Cameras or loop sensors on each lane, ambulance detector	Produce vehicle detections with lane, speed, type
Edge box	Object detection and tracking (for example YOLO + tracker)	Turn video into small event records
Ingestion	Kafka topic `lane_events`, **8 partitions** keyed by lane	Reliable, ordered stream per lane
Stream processing	Spark Structured Streaming on YARN	Per-lane 10 s counts, waiting time, ambulance flag, **all 8 lanes in parallel**
Decision	Cloudflare Worker + Durable Object + KV	Run rules R1 to R10, keep junction state, return recommendation
Storage	**HDFS** (raw, curated, results zones)	Keep all events and decisions for analysis

Batch analytics	**Hadoop MapReduce**, Spark, Hive	Highway analysis, threshold calibration, forecasting
Actuation	Local signal controller with safety interlock	Applies the recommendation safely
Dashboard	Cloudflare Pages or any web UI	Live lane counts, current green, decisions, alerts

4.2 Why this is “big data”

One junction with 8 lanes, about 30 tracked vehicles per lane and 10 frames per second produces roughly 2,400 records per second. At about 200 bytes per record that is about 0.5 MB/s, or about **40 GB per day for one junction**. For 100 junctions that is about 4 TB per day. These numbers are an illustrative estimate, but they show why a distributed file system (HDFS) and parallel engines (Hadoop, Spark) are needed once the system grows beyond a demo.

5. Phase 1: highway analysis in both directions

Goal: understand how traffic really behaves before we write rules. The highway study gives us the numbers for the junction rules (threshold `T`, minimum count `C_MIN`, peak hours).

5.1 Data and fields

Use a real data set (for example the Caltrans PeMS highway sensor data), a recorded highway video processed with a detector, or simulated data from SUMO. Each record:

`ts, direction (A or B), lane, speed, vehicle_type`

5.2 Metrics computed (separately for each direction)

Metric	Meaning	How
Flow `q`	Vehicles per hour	count per hour per direction
Speed `v`	Average speed	mean of `speed`
Density `k`	Vehicles per km	`k = q / v` (fundamental relation `q = k * v`)
Peak hour	Hour with the highest flow	top hour per direction
Directional split	Share of traffic in each direction	`q_A / (q_A + q_B)`
Congestion index	How slow the road is	`1 - v / v_free`
Vehicle mix	Cars, buses, trucks, bikes, ambulances	count by type
Day type	Weekday vs weekend pattern	group by day of week

5.3 Hadoop MapReduce job (hourly flow per direction)

Mapper (`mapper.py`):

```
#!/usr/bin/env python3
# input line: ts,direction,lane,speed,vehicle_type   (one record = one vehicle passing the sensor)
import sys
for line in sys.stdin:
    p = line.strip().split(",")
    if len(p) < 5 or p[0] == "ts":
        continue
    hour = p[0][11:13]                    # "2026-09-01T08:15:03" -> "08"
    print(f"{p[1]}|{hour}\t1\t{p[3]}")    # key = direction|hour
```

Reducer (`reducer.py`):

```
#!/usr/bin/env python3
import sys
cur, n, s = None, 0, 0.0
for line in sys.stdin:
    key, one, speed = line.rstrip("\n").split("\t")
    if key != cur and cur is not None:
        print(f"{cur}\t{n}\t{s / n:.1f}")     # key, vehicles per hour, average speed
        n, s = 0, 0.0
    cur = key
    n += 1
    s += float(speed)
if cur is not None:
    print(f"{cur}\t{n}\t{s / n:.1f}")
```

Run it on the cluster:

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
  -files mapper.py,reducer.py \
  -mapper "python3 mapper.py" -reducer "python3 reducer.py" \
```

```
  -input  /traffic/raw/highway/dt=2026-09-01 \
  -output /traffic/results/highway_hourly/dt=2026-09-01
```

HDFS splits the input into 128 MB blocks, YARN starts one mapper per block in parallel, and the reducer adds up each `direction|hour` key.

5.4 Hive queries for the report

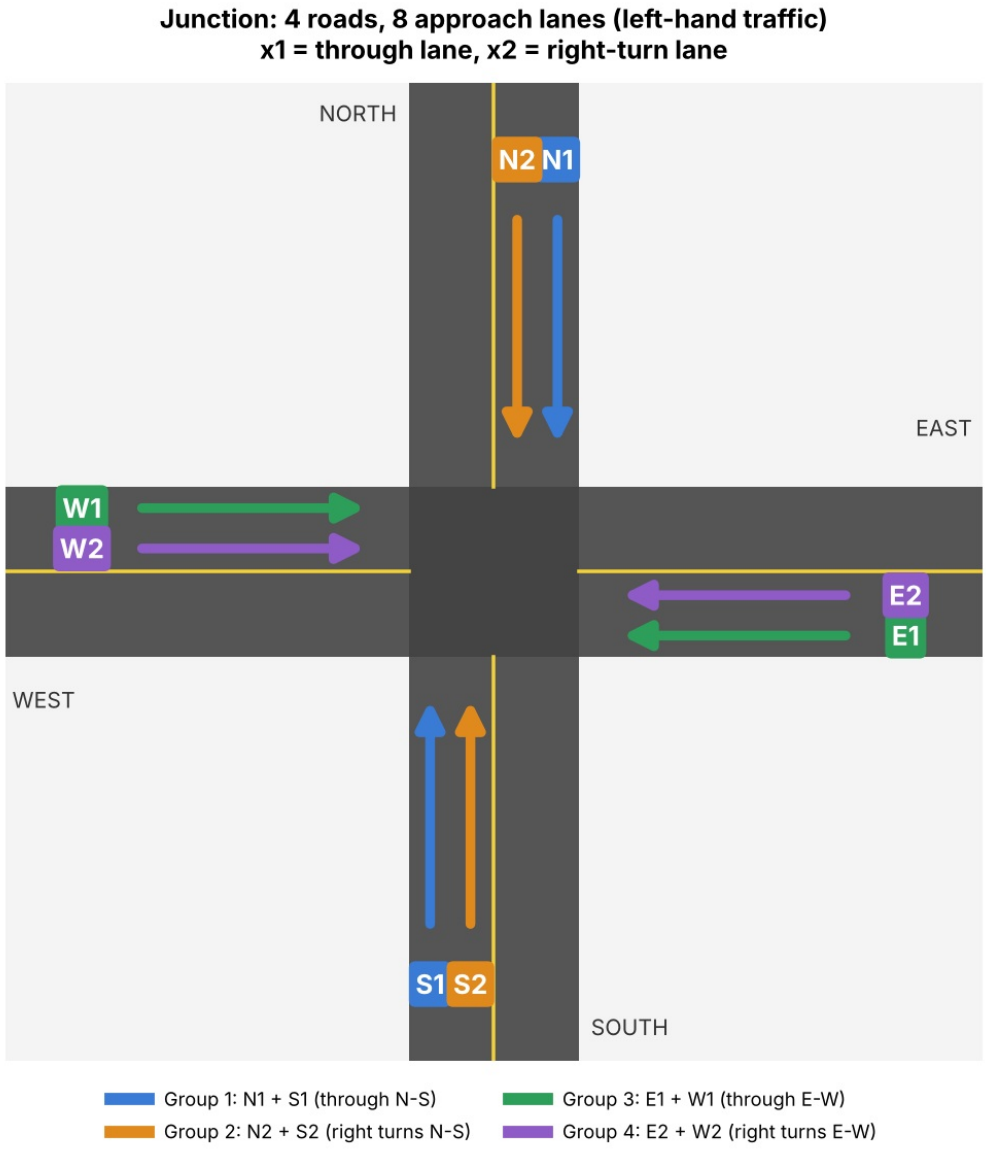
```
CREATE EXTERNAL TABLE highway_events (
  ts TIMESTAMP, direction STRING, lane INT, speed DOUBLE, vehicle_type STRING)
PARTITIONED BY (dt STRING)
STORED AS PARQUET
LOCATION '/traffic/curated/highway_events';
MSCK REPAIR TABLE highway_events;

-- flow and speed per direction per hour
SELECT direction, hour(ts) AS hr, COUNT(*) AS vehicles, ROUND(AVG(speed),1) AS avg_speed
FROM highway_events
WHERE dt BETWEEN '2026-09-01' AND '2026-09-30'
GROUP BY direction, hour(ts)
ORDER BY direction, vehicles DESC;
```

5.5 What Phase 1 gives to Phase 2

- **Threshold `T`:** take the 90th percentile of lane counts in peak hours and round to the nearest of 50 or 100. In Spark: `percentile_approx(count, 0.9)`.
- **Minimum count `C_MIN`:** about 10 percent of `T` (5 when `T = 50`).
- **Peak-hour profile:** used to build realistic arrival rates for the junction simulation.
- **Upstream effect:** if the highway feeds the junction, a rise in highway flow warns that queues will grow in the next minutes.

6. Phase 2: the junction model



Junction with 4 roads and 8 approach lanes, coloured by compatible group.

6.1 Lanes

Lane	Road	Movement
N1, S1, E1, W1	North, South, East, West	Through (and left turn)
N2, S2, E2, W2	North, South, East, West	Right turn

6.2 Green groups (which lanes can be green together)

A chosen lane can safely share the green with the opposite lane of the same type. This is how “maintain all lanes in parallel” is handled safely.

Group	Lanes	Why they do not conflict
G1	N1 + S1	Opposite through flows
G2	N2 + S2	Opposite right turns pass each other
G3	E1 + W1	Opposite through flows
G4	E2 + W2	Opposite right turns pass each other

If the selected lane’s partner has very little traffic, the partner stays red (rule R3, R6) and only the selected lane gets green.

6.3 What we track for every lane

Field	Meaning
`count`	Vehicles in the lane zone right now (10 s window)
`wait`	Seconds since this lane last had green while vehicles were waiting. Reset to 0 when the lane is green or empty
`ambulance`, `amb_conf`, `amb_eta`	Ambulance present, detection confidence, seconds until it reaches the stop line
`last_green_ts`	When the lane last had green

7. Data design and HDFS layout

7.1 Event record (Kafka and raw HDFS)

```
{"ts":"2026-10-24T19:33:10Z","intersection_id":"J1","lane_id":"N1",
 "track_id":"v-88231","vehicle_type":"car","speed_kmph":14.2,"confidence":0.91}
```

7.2 Lane-state record (output of Spark, input of the decision API)

```
{"intersection_id":"J1","ts":"2026-10-24T19:33:10Z",
 "lanes":[{"id":"N1","count":62,"avg_speed":9.5,"ambulance":false,"amb_conf":0.0,"amb_eta":999},
          {"id":"W2","count":8,"avg_speed":11.0,"ambulance":true,"amb_conf":0.93,"amb_eta":12}]}
```

7.3 HDFS folders

```
/traffic/raw/highway/dt=YYYY-MM-DD/          raw highway events
/traffic/raw/junction/dt=YYYY-MM-DD/         raw junction events (from Kafka)
/traffic/curated/lane_counts_10s/dt=.../     10-second lane counts (Parquet)
/traffic/curated/highway_events/dt=.../      cleaned highway events (Parquet)
/traffic/results/highway_hourly/             MapReduce / Hive outputs
/traffic/results/decisions/dt=.../           every recommendation with rule and reason
/traffic/models/                             forecast model files, calibrated parameters
```

Settings to mention in the report: block size 128 MB, replication factor 3 (fault tolerance), Parquet with Snappy compression, partition by date.

8. Parallel processing with HDFS, Hadoop and Spark

Technique	How it gives parallelism
Kafka partitions	8 partitions keyed by `lane_id`: each lane is its own ordered stream
Spark Structured Streaming	8 partitions become 8 tasks across executors, so all lanes are counted at the same time
HDFS blocks	A big file is split into 128 MB blocks stored on different DataNodes; many readers work at once
MapReduce / Spark batch	One mapper per block, shuffle by key, reducers combine results
YARN	Schedules the jobs across the cluster and restarts failed tasks
Replication (3x)	If a DataNode fails, another copy is used and processing continues

8.1 Spark Structured Streaming: per-lane 10-second features

```
from pyspark.sql import SparkSession, functions as F, types as T
import requests, json

spark = SparkSession.builder.appName("lane-state").getOrCreate()

schema = T.StructType([
    T.StructField("ts", T.TimestampType()),
    T.StructField("intersection_id", T.StringType()),
    T.StructField("lane_id", T.StringType()),
    T.StructField("track_id", T.StringType()),
    T.StructField("vehicle_type", T.StringType()),
    T.StructField("speed_kmph", T.DoubleType()),
    T.StructField("confidence", T.DoubleType()),
])

events = (spark.readStream.format("kafka")
          .option("kafka.bootstrap.servers", "kafka:9092")
          .option("subscribe", "lane_events").load()
          .select(F.from_json(F.col("value").cast("string"), schema).alias("e"))
          .select("e.*"))

lane_state = (events.withWatermark("ts", "30 seconds")
    .groupBy(F.window("ts", "10 seconds"), "intersection_id", "lane_id")
    .agg(F.approx_count_distinct("track_id").alias("count"),          # unique vehicles in the zone
```

```
            F.avg("speed_kmph").alias("avg_speed"),
            F.max(F.when(F.col("vehicle_type") == "ambulance", F.col("confidence"))
                  ).alias("amb_conf")))

    def send_to_worker(batch_df, batch_id):
        rows = [r.asDict() for r in batch_df.collect()]
        if rows:
            requests.post("https://<your-worker>.workers.dev/decide",
                          headers={"Authorization": "Bearer <TOKEN>"},
                          data=json.dumps(rows, default=str), timeout=3)

    (lane_state.writeStream.outputMode("update")
        .foreachBatch(send_to_worker)
        .option("checkpointLocation", "hdfs:///traffic/checkpoints/lane_state")
        .start().awaitTermination())
```

The same events are also written to HDFS (a second `writeStream` to Parquet) so the batch jobs can use them later.

9. Cloudflare decision layer

9.1 Role

The Worker is the **decision model**. It receives the 10-second lane state, runs rules R1 to R10, and returns a recommendation. It runs close to the junction at the edge, so the answer returns in well under a second.

Cloudflare part	Used for
Worker	`/decide` endpoint, authentication, runs the rule engine (a JavaScript port of the reference code in Section 13)
Durable Object (one per junction)	Keeps the junction state safely in one place: current green group, elapsed green time, `wait` counters, ambulance pre-emption flag
KV	Stores tuned parameters (`T`, `C_MIN`, weights) written by the Hadoop/Spark batch jobs
Queues + R2	Archive every decision as a log; R2 is S3-compatible, so Hadoop can copy the logs into HDFS with `distcp` over `s3a`
Pages	Dashboard

9.2 Worker skeleton

```
    export default {
      async fetch(request, env) {
        const url = new URL(request.url);
        if (url.pathname === "/decide" && request.method === "POST") {
          if (request.headers.get("Authorization") !== `Bearer ${env.API_TOKEN}`)
            return new Response("unauthorized", { status: 401 });

          const body = await request.json();                      // lane states from Spark
          const cfg = JSON.parse((await env.PARAMS.get(`cfg:${body.intersection_id}`)) ?? "{}");
          const stub = env.JUNCTION.get(env.JUNCTION.idFromName(body.intersection_id));
          const decision = await stub
            .fetch("https://do/decide", { method: "POST", body: JSON.stringify({ lanes: body, cfg }) })
            .then(r => r.json());

          await env.DECISION_LOG.send(decision);                  // Queue -> R2 -> HDFS
          return Response.json(decision);
        }
        return new Response("not found", { status: 404 });
      },
    };
```

9.3 Response format

```
    {"intersection_id":"J1","ts":"2026-10-24T19:33:10Z","action":"SWITCH",
     "green":["N1","S1"],"hold_s":10,"transition_s":5,
     "rule":"R5","reason":"N1 has the highest score 0.539","preempt":false}
```

9.4 Closing the learning loop

Each night a Spark job reads the logged decisions and lane counts from HDFS, recalculates `T`, `C_MIN` and the weights, trains an optional short-term forecast model (for example gradient-boosted trees in Spark MLlib to predict the next 5 minutes of counts), and writes the new values to Cloudflare KV. The Worker uses them from the next tick.

10. Parameters

All numbers below are configuration values (`CFG` in the code). Tune them with simulation.

Name	Default	Meaning

`TICK`	10 s	Decision interval. Signals change only on a tick
`T`	50 (or 100)	Heavy-lane threshold in vehicles
`C_MIN`	5	Below this a lane is “very low traffic” and is skipped
`W_MAX`	120 s	A lane waiting this long (with at least 1 vehicle) is opened and flushed
`BOOST`	1.25	Wait multiplier for a heavy lane (the “wait count becomes a little higher” rule)
`ALPHA`, `BETA`	0.6, 0.4	Weights of count and waiting time in the score
`C_CAP`, `W_CAP`	100, 120	Normalisation limits for count and wait
`MIN_GREEN`, `MAX_GREEN`	10 s, 40 s	Green time limits
`MARGIN`	1.15	A challenger must beat the current green by 15 percent
`FLUSH_PER_VEH`	2 s	Green seconds per vehicle in flush mode
`AMB_CONF`	0.80	Minimum ambulance detection confidence
`AMB_MAX_HOLD`	60 s	Longest green held for an ambulance
`FIXED_GREEN`	30 s	Per-group green in fail-safe mode
`YELLOW`, `ALL_RED`	3 s, 2 s	Safety clearance, never skipped

11. Decision rules and logic

11.1 Priority score

For every lane `i` on each tick:

```
heavy_i = 1 if count_i >= T else 0
W_eff_i = wait_i * BOOST            if heavy_i = 1
        = wait_i                    otherwise
score_i = ALPHA * min(count_i / C_CAP, 1) + BETA * min(W_eff_i / W_CAP, 1)
```

So a lane scores higher when it has more vehicles and when it has waited longer. Once the count crosses `T`, the wait part grows 25 percent faster. The heavy lane gets a better chance, but the signal does **not** jump immediately; it can only win on the next tick and only if the safety rules allow it.

11.2 The rules

ID	Rule	What it does
R1	**Ambulance pre-emption**	If an ambulance is detected (confidence at least 0.80) on a lane, switch to that lane’s group at once. Current green goes yellow (3 s), then all-red (2 s), then the ambulance lane gets green. Hold until the ambulance has passed the stop line, at most 60 s. If several ambulances, the lowest ETA goes first. If the ambulance’s lane is already green, just keep it
R2	**Starvation protection and flush**	If a lane has waited at least `W_MAX` (120 s) and has at least 1 vehicle, open it next. Green time = 2 s per vehicle (rounded up to the 10 s tick, between 10 and 40 s) so the lane is **emptied completely** even if it has few vehicles
R3	**Skip very low traffic**	A lane with `count < C_MIN` is not eligible for green. Empty lanes also reset their wait to 0, so they cannot “age” into false priority
R4	**Heavy-lane boost**	If `count >= T`, multiply the wait by `BOOST` in the score. No direct switching
R5	**Highest score wins**	Among eligible lanes, the highest score gets green. Ties: higher count, then longer wait, then fixed order N1, N2, E1, E2, S1, S2, W1, W2
R6	**Parallel green with opposite lane**	The opposite partner of the winner joins the

		green if it has at least `C_MIN` vehicles (or an ambulance). Otherwise it stays red
R7	**10-second tick and minimum green**	Decisions are made every 10 s. A green phase must run at least `MIN_GREEN` (and any flush time promised by R2). No mid-phase switching, except for R1
R8	**Hysteresis and maximum green**	Keep the current green unless a challenger scores more than 1.15 times the current best, or the current group reached `MAX_GREEN` (40 s). After 40 s the current lanes cannot be chosen again if any other lane is eligible. This stops rapid flipping
R9	**Fail-safe**	If data is stale (no update for 3 ticks, 30 s) or the sensors fail, ignore scores and run the fixed plan G1, G3, G2, G4 with 30 s each
R10	**Idle junction**	If every lane has fewer than `C_MIN` vehicles, keep the current green (rest in green) and do not change anything

11.3 Order in which the rules are checked

1. Safety timings: yellow (3 s) and all-red (2 s) are **never skipped**, not even for an ambulance.
2. **R9** fail-safe (if data cannot be trusted, nothing else can be). Ambulance pre-emption must then come from a separate local input such as a wired or radio ambulance signal at the controller.
3. **R1** ambulance.
4. **R7** minimum green or promised hold.
5. **R2** starvation flush.
6. **R3** skip low-traffic lanes (and **R10** if nothing is left).
7. **R8** maximum green, then **R4**/**R5** scoring and choice.
8. **R8** hysteresis check; if the challenger is not clearly better, keep the current green.
9. **R6** add the opposite partner, then output the decision.

11.4 Decision flow in words

```
every 10 s:
  if data stale                          -> R9 fixed plan
  if ambulance (conf >= 0.8)             -> R1: switch to its group (yellow + all-red first), hold until it passes
  if current green not yet at min hold   -> R7: keep
  if a lane waited >= 120 s and has >=1  -> R2: open it and flush it
  keep only lanes with count >= 5        -> R3
  if none left                           -> R10: rest in green
  score the lanes (heavy boost if >= T)  -> R4
  best = highest score                   -> R5
  if current group still good enough
     (best <= 1.15 x current)            -> R8: keep
  else give green to best + opposite lane if it has traffic -> R6
```

11.5 Ambulance priority in detail

Step	Action
Detect	Camera class “ambulance” with flashing-light check, siren audio, or a signal from the vehicle (GPS, RFID or radio). Require confidence of at least 0.80 for two readings in a row to avoid false alarms
Decide	Worker returns `preempt: true` with the group to open
Transition	If the ambulance lane is not green: yellow 3 s, all-red 2 s, then green. **“Instant” means the next safe moment, at most about 5 seconds**, never a sudden conflict
Hold	Keep green until the exit sensor sees the ambulance pass, plus 3 s, or 60 s at most
Several ambulances	Lowest ETA first. Ambulances on opposite lanes of the same group go together. A conflicting one waits
Recovery	Normal rules resume. The other lanes kept their wait counters, so they are served soon, and R2 protects any lane that waited too long
Logging	Every pre-emption is logged with time, lane, confidence and hold time; repeated pre-emptions on one lane raise an alert for the operator

12. Worked examples

Junction snapshot used below (current green is E1 + W1, running for 20 s):

Lane	Count	Wait (s)	Score	Remark
N1	62	40	**0.539**	Heavy (at least 50): wait boosted
N2	12	70	0.305	
E1	30	0 (green)	0.180	
E2	8	90	0.348	
S1	48	55	0.471	Just under the threshold
S2	3	20	0.085	**Skipped** (below `C_MIN`)
W1	35	0 (green)	0.210	
W2	0	0	0.000	**Skipped** (empty)

Score of N1: `0.6 * 0.62 + 0.4 * (40 * 1.25 / 120) = 0.372 + 0.167 = 0.539`.

Case	Change in the situation	Decision	Rule
A. Normal	Snapshot above	Switch to **N1 + S1**, hold 10 s (S1 joins because it has 48 vehicles)	R5, R6
B. Not direct	Same, but green just started (0 s)	Keep E1 + W1	R7
B2. Heavy rises	N1 jumps to 80 (score 0.647) right after a switch	Still keep E1 + W1; N1 can win on a later tick	R4, R7
C. Starvation	E2 has waited 125 s with 8 vehicles	Open **E2** only (W2 is empty), green 20 s so all 8 vehicles clear	R2
D. Ambulance	Ambulance on W2, confidence 0.93, ETA 12 s	Pre-empt: yellow, all-red, then **W2 + E2** green	R1
E. Sensor failure	Data not updated for 30 s	Fixed plan continues	R9
F. Skip	S2 has 3 vehicles and has waited 100 s (score would be 0.351)	S2 still skipped, N1 + S1 chosen. At 120 s rule R2 will open S2 once to empty it	R3, R2
G. Hysteresis	N1 = 42 (score 0.252) vs current E1 = 40 (0.240)	Keep: 0.252 is not above 0.240 x 1.15 = 0.276	R8
H. Max green	N1 + S1 have had 40 s	Switch to the next best lane, **E2** (0.348)	R8, R5

These results come from running the reference code in Section 13 on the snapshot above.

13. Reference implementation (Python)

This is the decision engine used for the examples. The Cloudflare Worker uses the same logic in JavaScript, and the same file can be used as a unit-test target and inside the simulation.

```
        """
        decision_engine.py - Adaptive traffic-signal decision logic (reference implementation)

        Junction with 4 roads and 8 approach lanes:
            N1 N2 E1 E2 S1 S2 W1 W2
            x1 = through lane of that road
            x2 = turning lane that crosses the opposite flow (right turn, left-hand traffic)

        Two lanes can be green together ("a group") when they are the opposite pair:
            N1+S1, N2+S2, E1+W1, E2+W2
        """
        import math
        from dataclasses import dataclass

        LANES = ["N1", "N2", "E1", "E2", "S1", "S2", "W1", "W2"]
        PARTNER = {"N1": "S1", "S1": "N1", "N2": "S2", "S2": "N2",
                   "E1": "W1", "W1": "E1", "E2": "W2", "W2": "E2"}
        FIXED_PLAN = [("N1", "S1"), ("E1", "W1"), ("N2", "S2"), ("E2", "W2")]

        CFG = {
            "TICK": 10,           # decision interval (seconds)
            "T": 50,              # heavy-traffic threshold (vehicles in lane zone)
            "C_MIN": 5,           # below this a lane is "very low" and is skipped
            "W_MAX": 120,         # starvation limit (seconds without green)
            "BOOST": 1.25,        # wait multiplier for a heavy lane (count >= T)
            "ALPHA": 0.6,         # weight of vehicle count in the score
            "BETA": 0.4,          # weight of waiting time in the score
            "C_CAP": 100,         # count normalisation cap
            "W_CAP": 120,         # wait normalisation cap
            "MIN_GREEN": 10,      # minimum green (seconds)
            "MAX_GREEN": 40,      # maximum green (seconds)
            "MARGIN": 1.15,       # a challenger must beat the incumbent by 15 %
```

```
    "FLUSH_PER_VEH": 2,   # seconds of green per vehicle in flush mode
    "AMB_CONF": 0.80,     # minimum detection confidence for an ambulance
    "AMB_MAX_HOLD": 60,   # maximum ambulance green hold (seconds)
    "FIXED_GREEN": 30,    # per-group green in fail-safe mode (seconds)
    "YELLOW": 3,
    "ALL_RED": 2,
}


@dataclass
class Lane:
    id: str
    count: int = 0            # vehicles currently in the lane's monitoring zone
    wait: float = 0.0         # seconds since last green while vehicles waited
    ambulance: bool = False
    amb_conf: float = 0.0
    amb_eta: float = 999.0    # seconds until the ambulance reaches the stop line


@dataclass
class State:
    green: tuple = ()         # lanes that are green right now
    green_elapsed: float = 0.0
    hold: float = 0.0         # green time promised by the last decision
    preempt: bool = False


def score(lane, cfg=CFG):
    """Priority score in 0..1. Heavy lanes (count >= T) get a boosted wait value."""
    c = min(lane.count / cfg["C_CAP"], 1.0)
    boost = cfg["BOOST"] if lane.count >= cfg["T"] else 1.0
    w = min(lane.wait * boost / cfg["W_CAP"], 1.0)
    return cfg["ALPHA"] * c + cfg["BETA"] * w


def _group(lanes, first, cfg):
    """The chosen lane plus its opposite partner (only if the partner has traffic)."""
    partner = lanes[PARTNER[first]]
    if partner.count >= cfg["C_MIN"] or partner.ambulance:
        return (first, partner.id)
    return (first,)


def _out(action, green, hold, rule, why, preempt=False, transition=0):
    return {"action": action, "green": tuple(green), "hold": hold,
            "rule": rule, "why": why, "preempt": preempt, "transition": transition}


def decide(lanes, st, data_ok=True, cfg=CFG):
    """Called once every TICK seconds. lanes: dict id -> Lane."""
    order = {k: i for i, k in enumerate(LANES)}

    # R9  Fail-safe: stale or missing data -> fixed-time plan
    if not data_ok:
        if st.green and st.green_elapsed < cfg["FIXED_GREEN"]:
            return _out("KEEP", st.green, cfg["TICK"], "R9", "data stale, fixed plan running")
        idx = next((i for i, g in enumerate(FIXED_PLAN) if st.green and st.green[0] in g), -1)
        nxt = FIXED_PLAN[(idx + 1) % len(FIXED_PLAN)]
        return _out("SWITCH", nxt, cfg["FIXED_GREEN"], "R9", "data stale, next fixed group",
                    transition=cfg["YELLOW"] + cfg["ALL_RED"])

    # R1  Ambulance pre-emption (highest priority)
    amb = [l for l in lanes.values() if l.ambulance and l.amb_conf >= cfg["AMB_CONF"]]
    if amb:
        target = min(amb, key=lambda l: l.amb_eta)
        if target.id in st.green:
            if st.green_elapsed < cfg["AMB_MAX_HOLD"]:
                return _out("KEEP", st.green, cfg["TICK"], "R1",
                            f"ambulance on {target.id}, lane already green", preempt=True)
        else:
            return _out("SWITCH", _group(lanes, target.id, cfg), cfg["TICK"], "R1",
                        f"ambulance on {target.id} (ETA {target.amb_eta:.0f}s): pre-empt",
                        preempt=True, transition=cfg["YELLOW"] + cfg["ALL_RED"])

    # R7  Minimum green / promised hold must be respected
    if st.green and st.green_elapsed < max(cfg["MIN_GREEN"], st.hold):
        return _out("KEEP", st.green, cfg["TICK"], "R7", "minimum green not finished")

    # R2  Starvation: a lane waited too long -> open it and flush it
    starved = [l for l in lanes.values()
               if l.wait >= cfg["W_MAX"] and l.count >= 1 and l.id not in st.green]
    if starved:
        s = max(starved, key=lambda l: (l.wait, l.count))
        flush = math.ceil(s.count * cfg["FLUSH_PER_VEH"] / cfg["TICK"]) * cfg["TICK"]
        flush = max(cfg["MIN_GREEN"], min(cfg["MAX_GREEN"], flush))
        return _out("SWITCH", _group(lanes, s.id, cfg), flush, "R2",
                    f"{s.id} waited {s.wait:.0f}s: flush {s.count} vehicles",
                    transition=cfg["YELLOW"] + cfg["ALL_RED"])

    # R3  Skip lanes with very low traffic
    eligible = [l for l in lanes.values() if l.count >= cfg["C_MIN"]]
    if not eligible:
        return _out("KEEP", st.green, cfg["TICK"], "R10", "all lanes below C_MIN, rest in green")

    # R8  Max green reached -> incumbent lanes may not be chosen again
    if st.green and st.green_elapsed >= cfg["MAX_GREEN"]:
        others = [l for l in eligible if l.id not in st.green]
        candidates = others or eligible
    else:
        candidates = eligible
```

```
        # R4 + R5  Score (with heavy boost) and pick the highest; ties by count, wait, order
        best = max(candidates, key=lambda l: (round(score(l, cfg), 6), l.count, l.wait, -order[l.id]))

        # R8  Hysteresis: keep the current group unless the challenger is clearly better
        current = [lanes[i] for i in st.green if lanes[i].count >= cfg["C_MIN"]]
        if current and st.green_elapsed < cfg["MAX_GREEN"]:
            cur_best = max(score(l, cfg) for l in current)
            if best.id in st.green or score(best, cfg) <= cur_best * cfg["MARGIN"]:
                return _out("KEEP", st.green, cfg["TICK"], "R8", "incumbent still best (hysteresis)")

        # R6  Chosen lane + opposite partner get green
        return _out("SWITCH", _group(lanes, best.id, cfg), cfg["TICK"], "R5",
                    f"{best.id} has the highest score {score(best, cfg):.3f}",
                    transition=cfg["YELLOW"] + cfg["ALL_RED"])


    def apply(lanes, st, d, cfg=CFG):
        """Apply a decision and advance one TICK (updates wait counters)."""
        if d["action"] == "SWITCH":
            st.green, st.green_elapsed, st.hold, st.preempt = d["green"], 0.0, d["hold"], d["preempt"]
        for l in lanes.values():
            if l.id in st.green:
                l.wait = 0.0
            elif l.count > 0:
                l.wait += cfg["TICK"]
            else:
                l.wait = 0.0
        st.green_elapsed += cfg["TICK"]
```

14. Safety, reliability and security

- **Conflict monitor:** the local controller must refuse any command that would turn two conflicting lanes green. The cloud only recommends; the local hardware enforces.
- **Yellow and all-red are mandatory** between phases, also for ambulances.
- **Fail-safe:** if the cloud, the network or the sensors fail, the local controller runs the fixed-time plan (R9).
- **Latency budget:** detection to decision should finish in about 2 to 3 seconds so that the decision is ready before the next 10-second tick.
- **Security:** API token per junction, HTTPS only, rate limiting and firewall rules on the Worker, signed recommendations that the controller verifies, no personal data stored (no number plates, only anonymous track IDs).
- **Audit trail:** every decision is stored with its rule number and reason, so any switching can be explained afterwards.

15. Testing and evaluation

15.1 Simulation set-up

Use SUMO (open-source traffic simulator, which supports emergency vehicles) or a Python generator with Poisson arrivals and time-of-day rates taken from Phase 1. Compare the **adaptive system** with a **fixed-time baseline** (30 s per group).

Scenario	Purpose
Balanced traffic	Check the system is not worse than fixed time
Rush hour, one heavy road	Check heavy-lane boost and no direct switching
Night, very low traffic	Check skip rule (R3) and rest in green (R10)
One lane with a few vehicles for a very long time	Check starvation flush (R2)
Ambulance arrives from each of the 8 lanes	Check pre-emption time and recovery (R1)
Sensor and network failure	Check fail-safe (R9)

15.2 Metrics

Average waiting time per vehicle, maximum waiting time of any lane, average queue length, throughput (vehicles per hour), number of signal changes per hour, ambulance delay (seconds from detection to green), and processing latency of the pipeline. Report the percentage change against the fixed-time baseline using the numbers you measure.

15.3 Unit tests

Each row of the worked-example table in Section 12 is one test case with a fixed input and an expected output. Add tests for ties, a lane exactly at `T` and `C_MIN`, two ambulances at once, and a lane that reaches 120 s while an ambulance is passing.

15.4 Parameter study

Vary `T` (50 and 100), the weights `ALPHA` and `BETA`, `MARGIN` and `MAX_GREEN`, and show how average and maximum waiting time change.

16. Technology stack

Area	Tools
Big data	Hadoop 3.x (HDFS, YARN, MapReduce), Spark 3.x, Hive, Kafka
Language	Python (PySpark, reference engine), JavaScript or TypeScript (Worker)
Edge and cloud	Cloudflare Workers, Durable Objects, KV, Queues, R2, Pages
Vision (optional)	YOLO-based detector with a tracker, OpenCV
Simulation	SUMO (TraCI) or a Python traffic generator
Data	PeMS highway data or generated data; recorded video
Visualisation	Dashboard (web page), Hive or Spark results as tables and charts
Deployment for the demo	Docker Compose with 1 NameNode and 2 to 3 DataNodes on a laptop, or a small cloud cluster

17. Project plan (6 weeks)

Week	Work	Output
1	Study, set up Hadoop/HDFS/Spark/Kafka, collect or generate highway data	Working cluster, data in HDFS
2	Phase 1: MapReduce and Hive analysis in both directions	Flow, speed, peak-hour tables and charts; chosen `T` and `C_MIN`
3	Junction simulation and lane-count pipeline (Kafka, Spark streaming)	Live 10 s lane state for 8 lanes
4	Decision engine and Cloudflare Worker, parameters in KV	Recommendation every 10 s
5	Ambulance, starvation, fail-safe, dashboard, logs to HDFS	All rules R1 to R10 working
6	Experiments against fixed-time, parameter study, report and presentation	Final report, demo, slides

18. Expected deliverables

1. Source code: MapReduce jobs, Spark streaming job, decision engine, Worker, simulation scripts.
2. Highway analysis report (both directions) with tables and charts.
3. Running demo: 8-lane live counts, current green, recommendation and reason on a dashboard.
4. Evaluation report: adaptive vs fixed-time results and the parameter study.
5. This project document, and a short presentation.

19. Risks, limitations and future work

Risk or limit	Reduction
Wrong counts from camera (night, rain, occlusion)	Use confidence filtering, smoothing over the 10 s window, and the fail-safe rule
False ambulance detection	Two readings in a row, confidence limit, operator alert, short hold
Network or cloud delay	Local controller keeps the last safe plan and the fixed-time fallback
Fixed thresholds do not fit every junction	Thresholds are recalibrated from HDFS data every night
Demo data is small	Show the scaling estimate from Section 4.2 and test on a larger generated data set
A prototype is not certified for roads	Present it as a simulation and recommendation system only

Future work: coordinating several junctions (green wave), pedestrian phases, bus priority, a learned policy (reinforcement learning) that is checked against the rule engine, and predicting congestion from highway flow upstream.

20. Short glossary

Term	Meaning
HDFS	Hadoop Distributed File System: splits files into blocks and stores

	copies on many machines
MapReduce	Batch model: map (emit key and value) then reduce (combine per key)
YARN	Cluster resource manager that schedules jobs
Spark Structured Streaming	Stream processing with windows, used for 10-second lane counts
Kafka	Message queue that keeps ordered event streams in partitions
Hive	SQL layer over files in HDFS
Worker	Small program that runs on Cloudflare’s edge network
Durable Object	A Cloudflare object with its own storage that keeps state for one junction
Pre-emption	Overriding the normal signal plan for an emergency vehicle
Starvation	A lane that never gets green because others always win
Hysteresis	Extra margin needed before changing a decision, to avoid flip-flopping